Containerisation & deployment of the cardiac pipeline

Complete guide — from the basic concepts to ready-to-use files

0. Read this first (summary)

This document explains, **from scratch**, how to package the project with **Docker** so it runs reproducibly, and then what a real **online deployment** would involve. It is organised in three parts:

- **Part A — the concepts** to understand before touching Docker.
- **Part B — this project's specifics**: why it is special (Ollama, GPU, large datasets, clinical data) and what goes where.
- **Part C — the concrete steps**: prerequisites, exact file contents, commands, verification, troubleshooting, and the limits of a public deployment.

Key message to keep in mind throughout: Docker makes the **environment** reproducible and launchable with one command. It does NOT remove the project's heavy requirements (the MedGemma model via Ollama, a GPU, ~8.5 GB of datasets) — it **organises** them. A deployment to a public URL, by contrast, hits real walls (RAM/GPU, cost, security, clinical data) and is out of scope for a prototype.

Part A — The concepts to understand first

A.1 The problem Docker solves

Today, running the project requires installing, on YOUR machine and in the right order: Python, the dependencies from `requirements.txt`, Ollama, the `medgemma:4b` model, and having the datasets in the right place. A colleague who clones the repository has to redo everything — and if their Python or library version differs, "it works on my machine" becomes "it doesn't work on theirs".

Docker packages the application AND its environment (the right Python, the right library versions, the system settings) into a single, reproducible object. Result: the same thing runs identically on any machine that has Docker.

A.2 Image vs container (the analogy to remember)

Two words come up constantly; you must tell them apart:

- An **image** = a *frozen, read-only template* (like a blueprint, or a class in programming). It contains the minimal system + Python + your dependencies + your code.
- A **container** = a *running instance*, created from an image (like an object instantiated from a class). You can launch several containers from the same image.

You **build** an image once (`docker build`), then **run** containers from it (`docker run`).

A.3 Dockerfile and docker-compose

- The `Dockerfile` is the *recipe* for the image: "start from a Python 3.11, install these libraries, copy this code, run this command". `docker build` reads this file and produces the image.
- The `docker-compose.yml` orchestrates **several containers** that work together, with their settings (ports, volumes, environment variables, network). Our project needs **two** containers — the app and Ollama — so `docker-compose` is the natural tool here.

A.4 Volumes: where the data goes

An image must stay **lightweight**. You NEVER put the large datasets in it. Instead you use a **volume**: a folder on your machine "mounted" into the container at startup. The container *sees* the data without it being copied into the image.

Two benefits: the image stays small and shareable, and the data **survives** the deletion/rebuild of a container (also useful for the models downloaded by Ollama).

A.5 Ports and networking

A container is isolated. To reach it from your browser, you **publish** a port: `-p 8501:8501` "links" port 8501 of the container to port 8501 of your machine → you open `http://localhost:8501`.

Between containers of the same `docker-compose`, they reach each other **by their service name** (e.g. the app contacts Ollama at `http://ollama:11434`, not `localhost`). This detail matters for this project (see C.0).

A.6 Local vs Docker vs Deployment

	Local (current)	Docker	Deployment (public URL)
Reproducible	No	**Yes**	Yes
Shareable	No	**Yes** (image + compose)	Yes (but heavy hosting)
Effort	Already done	Medium	High
Cost	Free	Free	Paid (GPU/RAM)
Fits this project	Yes	**Yes — the right next step**	No (real walls, see Part C.8)

Part B — Why this project is a special case

B.1 Ollama / MedGemma = a separate service

The project does not call a model "inside" the app: it calls **Ollama**, a separate server that hosts

MedGemma 4B and answers on port 11434. In Docker, this becomes **a second container** (`ollama`) that the app

contacts over the network. The model itself (~5 GB) is NOT in the image: you download it once into the Ollama

container (`ollama pull medgemma:4b`), stored in a volume.

B.2 GPU / CUDA

The ECG embedding and the MRI pipeline (YOLO + U-Net + classifier) use **PyTorch**, which is far faster on a

GPU. For a container to see your NVIDIA card, you need the **NVIDIA Container Toolkit** and to launch with

GPU access. Without a GPU, everything still works on **CPU**, just more slowly.

B.3 The datasets are large

Measured on the project:

Folder	Size	In the image?
`kaggle` (PTB-XL)	3.1 GB	No → **volume**
`physionet` (verification copy)	3.1 GB	No → **exclude** (duplicate, only used for the integrity test)
`ACDC` (MRI)	2.3 GB	No → **volume**
`rag_sources`	35 MB	Volume (or image)
`RTLP`	14 MB	Volume
`uci_heart_disease`	20 KB	Image OK
Model checkpoints (U-Net, classifier, YOLO, embedding, RAG index)	~215 MB total	**Image OK**

To remember: ~**8.5 GB of datasets** stay **out of the image** (mounted as volumes); the **~215 MB of checkpoints **can go in the image; the** MedGemma model (~5 GB)** is handled separately by Ollama.

B.4 Clinical data & fake "authentication"

The data here is **public** (PTB-XL, ACDC, UCI), so fine for a prototype. But the app has a role selector with

demo passwords (`patient2026`…) that are **NOT security**: it is a simple display *switch*. On a public URL

this would be unacceptable — and a real cardiac app handling patient data would fall under GDPR (encryption,

certified health hosting, etc.). This is one of the reasons this project stays **local**.

B.5 Summary: what goes where

- **In the image**: the code (`src/`, `views/`, `Home.py`, `ui_*.py`), the Python dependencies, the **checkpoints** (~215 MB), the small data (`uci_heart_disease`).
- **As a volume (mounted at launch)**: `kaggle/`, `ACDC/`, `rag_sources/`, `RTLP/`, and the Ollama models.
- **Separate service**: Ollama + MedGemma (2nd container, model downloaded into a volume).
- **Excluded**: `physionet/` (verification duplicate), caches, large regenerable results.

Part C — The concrete steps

C.0 One small code change required (do once)

In the code, Ollama's address is **hard-coded**: `host="http://localhost:11434"` (`src/agent_common.py`, `generate_report_medgemma`). In Docker, the app must reach the Ollama container at `http://ollama:11434` — not `localhost`. So this address must be made **configurable via an environment variable**, keeping `localhost` as the default (so the current local usage is **unchanged**):

```
# In src/agent_common.py — make the host configurable, default unchanged
import os
def generate_report_medgemma(
    prompt, model="medgemma:4b",
    host=os.environ.get("OLLAMA_HOST", "http://localhost:11434"),
    images=None, temperature=None, seed=None, num_predict=2048):
    ...
```

Then the `docker-compose` will set `OLLAMA_HOST=http://ollama:11434` for the app container. This is the same "additive, opt-in, no-op default" pattern used throughout the project.

C.1 Prerequisites to install (once)

1. **Docker Desktop** (Windows/Mac) or Docker Engine (Linux) — provides `docker` and `docker compose`.
2. **(GPU, optional) NVIDIA Container Toolkit** — so containers can see the NVIDIA card. Without it, everything runs on CPU.
3. The **datasets** already present in the repo (`kaggle/`, `ACDC/`, …) — they are mounted as volumes, not copied.

C.2 `.dockerignore` file (at the root)

Stops Docker from copying the large folders and caches into the image (otherwise slow build and huge image):

```
# Data mounted as volumes, never in the image
kaggle/
ACDC/
RTLP/
physionet/
rag_sources/
# Caches & regenerable artifacts
**/__pycache__/
*.pyc
.git/
.ipynb_checkpoints/
results/ui_sessions/
results/_backups/
archive/
# Local environments & secrets
.venv/
venv/
.env          # the Groq API key lives here — never bake it into the image
```

C.3 `Dockerfile` (at the root)

Recipe for the app image (Streamlit + code + dependencies + checkpoints):

```
FROM python:3.11-slim

WORKDIR /app

# No apt: ultralytics pulls opencv-python (needs libGL/libglib via apt), but many
# restricted networks block apt's HTTP mirrors (port 80). opencv-python-headless
# removes that system-lib need, so the build uses ONLY pip over HTTPS (443).
# CPU-only torch is installed first (from PyTorch's CPU index) so the default CUDA
# torch (~2-3 GB of unused NVIDIA wheels) is NOT pulled — keeps the image ~3.5 GB
# instead of ~8 GB. --mount=type=cache keeps pip wheels across build retries.
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install torch --index-url https://download.pytorch.org/whl/cpu
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt \
    && pip uninstall -y opencv-python opencv-python-headless || true
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install opencv-python-headless

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "Home.py", \
     "--server.address=0.0.0.0", "--server.port=8501", "--server.headless=true"]
```

> **Validated.** This exact Dockerfile builds a **3.58 GB** image and the container serves the app

> (`/_stcore/health` → HTTP 200). The torch inside the container runs on **CPU** — which is fine for Option A

> (below), where the GPU work is done by the host's native Ollama.

C.4 `docker-compose.yml` file (at the root)

Three run options — the built/pushed image is IDENTICAL in all three, only the run config differs:

- **Option A (chosen, validated)** — containerize the **app only**; it talks to the **native host Ollama**

(which already uses your GPU) via `host.docker.internal`. No NVIDIA Container Toolkit, LLM stays GPU-fast,

the container's torch runs CPU (fast enough, cached). **Keep native Ollama running before `up`.**

- **Option B** — everything containerized (app + an `ollama` service) with GPU passthrough. Needs the NVIDIA

Container Toolkit (+ WSL2-GPU on Windows). "Purest" but finicky, and it still needs datasets mounted + the

model pulled separately.

- **Option C** — both containers on CPU. No toolkit, but the LLM is too slow (30 s–2 min/call) — smoke-test only.

The `docker-compose.yml` below is **Option A**:

```
services:
  app:
    build: .
    image: cardiac-pipeline:local     # local tag; re-tag with your Docker Hub user to push
    ports:
      - "8501:8501"
    environment:
      # The app reaches the HOST's Ollama. Docker Desktop resolves host.docker.internal
      # automatically; extra_hosts covers Linux too.
      - OLLAMA_HOST=http://host.docker.internal:11434
      - GROQ_API_KEY=${GROQ_API_KEY:-}    # optional cloud backend (from .env / host env)
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      # Datasets mounted read-only (kept out of the image)
      - ./kaggle:/app/kaggle:ro
      - ./ACDC:/app/ACDC:ro
      - ./rag_sources:/app/rag_sources:ro
      - ./RTLP:/app/RTLP:ro
```

For **Option B/C**, add back an `ollama` service (`image: ollama/ollama`), set

`OLLAMA_HOST=http://ollama:11434`, add `depends_on: [ollama]`, and (Option B) a `deploy.resources` GPU block

on both services.

C.4b Optional: the Groq cloud backend (NOT a container)

The app offers a second, optional LLM backend — **Groq** (a stronger text model, `gpt-oss-120b`). Unlike

Ollama/MedGemma, **Groq is a hosted cloud API** (Groq's servers), so:

- **It is NOT a container** and needs **no service** in `docker-compose`. There is nothing to install or run for it.
- The app reaches it over the **internet** (containers have outbound network by default) by calling `https://api.groq.com/…`.
- The only thing needed is the `GROQ_API_KEY` environment variable, passed to the `app` container. The code already reads it from the environment (`os.environ.get("GROQ_API_KEY")`) — **no code change** (unlike

`OLLAMA_HOST`).

How to provide the key — put it in a `.env` file next to `docker-compose.yml` (Docker Compose loads it automatically and substitutes `${GROQ_API_KEY}`):

```
# .env  (do NOT commit this file)
GROQ_API_KEY=gsk_your_key_here
```

Two important points:

- **Optional / opt-in.** The default backend stays **local MedGemma** (the Ollama container). If you never select Groq in the UI and set no key, everything works offline — Groq is simply unused.
- **Text-only + data leaves the machine.** Groq is text-only (it cannot see the MRI image, so the MRI agent and the chat stay on local MedGemma), and choosing it sends the prompt to Groq's servers — fine for the public PTB-XL/ACDC/UCI data, not for private data.

So, to answer "separate container or how do we access it": **no container — just an API key + internet.** In Option A the only container is the **app** itself (local MedGemma runs as the native host Ollama, and Groq is a cloud API).

C.5 Launch, step by step

Option A — the native host Ollama provides the model, so there is **no** `ollama pull` **inside a container**; just make sure your native Ollama is running (it already has `medgemma:4b`).

```
# 1) Build the app image
docker compose build

# 2) (optional) publish to Docker Hub
docker tag cardiac-pipeline:local YOUR_USER/cardiac-pipeline:latest
docker login
docker push YOUR_USER/cardiac-pipeline:latest

# 3) Run locally (keep your native Ollama running)
docker compose up -d

# 4) (optional) follow the app logs
docker compose logs -f app

# 5) stop
docker compose down
```

C.6 Verify it works

- **Health:** `curl http://localhost:8501/_stcore/health` → `200` (validated). Or open `http://localhost:8501` → the role login page.
- **Container reaches host Ollama:**

`docker compose exec app python -c "import urllib.request,json; print([m['name'] for m in json.load(urllib.request.urlopen('http://host.docker.internal:11434/api/tags'))['models']])"`

→ should list `medgemma:4b` (validated).

- **End-to-end:** role *researcher* → ECG page → enable the LLM findings → a result comes back.

C.7 Troubleshooting (problems actually hit while building this)

Symptom	Likely cause	Fix
`apt-get ... Unable to connect to deb.debian.org` (port 80)	Restricted network (school/proxy) blocks HTTP	**Already handled**: the Dockerfile uses **no apt** (opencv-python-headless) — pip over HTTPS/443 only. Confirmed apt stays blocked even after a WSL restart.
Build pulls GBs of `nvidia_*`/`cuda_*` wheels, image ~8 GB	Default PyPI torch is the **CUDA** build	**Already handled**: the Dockerfile installs **CPU torch** first from `download.pytorch.org/whl/cpu`. Option A runs torch on CPU anyway.
`rpc error: ... reading from server: EOF` mid-build	Docker Desktop / WSL2 daemon hiccup	Quit Docker Desktop → `wsl --shutdown` → restart it. The `pip` cache mount means the retry resumes from downloaded wheels.
The app can't reach the model	native Ollama not running / `OLLAMA_HOST` wrong	Start your host Ollama; check `OLLAMA_HOST=http://host.docker.internal:11434`.
Huge image / very slow build	Datasets copied into the image	Check the `.dockerignore` (C.2) — training datasets & `physionet/` are excluded.
Datasets not found in the app	Volume mounted wrong	Check the `./kaggle:/app/kaggle` etc. paths.

C.8 Going further: online deployment (the real walls)

Putting the app on a **public URL** hits concrete obstacles for THIS project:

- **LLM RAM/GPU**: MedGemma 4B via Ollama needs ~5–6 GB of RAM (and a GPU to be smooth). Free hosts

(Streamlit Cloud, free Hugging Face Spaces) **cannot**; a GPU instance is **paid**.

- **~8.5 GB of datasets** to copy/mount on the server.
- **Security**: the role passwords are **fake** → unacceptable in public as-is.
- **Concurrency**: a single shared Ollama → queueing if several users.

"Realistic cloud" option: switch the whole LLM to the **free Groq cloud backend** (already supported,

`GROQ_API_KEY`), which removes the Ollama/GPU need… **but breaks multimodality**: the MRI agent and the chat

that "see" the image rely on local MedGemma (Groq is *text-only*). It's a trade-off, not a full solution.

Recommendation: aim for **Docker locally** (ideal for the demo/defense: reproducible, one command), and

consider public deployment **out of scope** for this prototype.

C.9 Final checklist

- [ ] Docker Desktop installed (no NVIDIA toolkit needed for Option A).
- [ ] Change C.0 applied (`OLLAMA_HOST` configurable) — done.
- [ ] `.dockerignore`, `Dockerfile`, `docker-compose.yml` at the root — done.
- [ ] `docker compose build` succeeds (image ~3.5 GB) — done & validated.
- [ ] Native host Ollama running with `medgemma:4b` (Option A) — validated from the container.
- [ ] `docker compose up -d` → `http://localhost:8501` health 200 — validated.
- [ ] (optional) tagged + pushed to Docker Hub.
- [ ] `physionet/` and the large folders correctly excluded from the image.

Document generated for the "cardiac-multiagent-pipeline" project (internship 25-26). Sizes and paths reflect the actual state of the repository at the time of writing.

docker start cardiac-multiagent-pipeline-app-1